

PyQCU 多重网格算法实现

CUDA C++ 版：求解器族、递归 Cycle、混合精度与分布式粗格

代码审计与算法验证

PyQCU · Lattice QCD

2026-08-28

结论先行：请求的 CUDA C++ 多重网格功能已形成可运行闭环

已完成

Chebyshev、CA-GCR、固定 $L = 2$ BiCGStabL；
V/W/F/K-cycle；逐层
c64/c128；32 方向分布式粗格；
device-aware MPI、重叠路径、
可选 NVSHMEM；五项 C++
verify()。

直接证据

大格 $16 \times 32 \times 32 \times 48$ 上新求解器的 full-op 真残差均低于 8.1×10^{-7} ；3-level W/F/K 小格均通过；五项 verify 返回 status=0。

当前边界

确证 正确性已测；**推断** 性能收益尚未证明。NVSHMEM 专用编译环境与 device-aware 实际 MPI 能力在当前机器未实测，不能外推吞吐。

阶段判断： 代码层面已经覆盖本轮指定的算法与通信接口；验收重点从“有没有实现”转为“健康 CUDA/MPI 平台上的性能、任意 L 与更大层次的系统性复测”。

来源：实现：cpp/cuda/qcu/include/lattice_clover_multigrid.h、lattice_multigrid.h、lattice_mpi.h；结果：pyqcu/testing/qcu/strict/quda_comparison/out/qcu_mg_verify*.json；一次性 agent 会话记录已清理。

问题与验收口径：粗网格必须同时保持物理算子和全局数值语义

$$\begin{aligned} D &= S + H, \\ S_{\text{Schur}} &= D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo}, \\ A_c &= R S_{\text{Schur}} P \simeq P^\dagger S_{\text{Schur}} P. \end{aligned}$$

物理图像

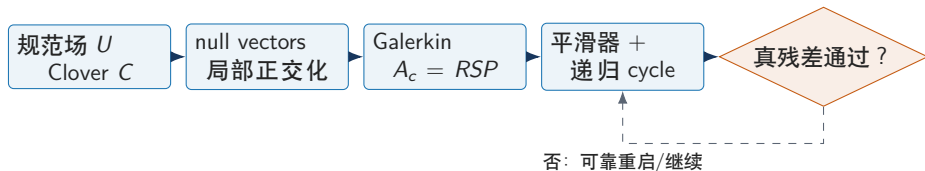
平滑器压低高频误差；近零空间将低频误差压到粗格；Galerkin 粗算子保留 spin-color 耦合和规范场背景。由于 Schur 算子跨两次 hopping，粗格是“on-site + 8 nearest + 24 diagonal”的 33 点模板。

验收量	通过含义
$\ RP\eta - \eta\ /\ \eta\ $	粗空间投影不漂移
$\ \langle P_c, v \rangle - \langle c, Rv \rangle\ $	transfer 伴随关系
$\ A_c\eta - RSP\eta\ /\ A_c\eta\ $	Galerkin 一致性
Schur 两 parity block	预条件算子确实实现
$\langle x, S^\dagger Sx \rangle = \ Sx\ ^2$	normal operator 正性
full-op $\ b - Dx\ /\ b\ $	生产求解真实残差

统一主测例： $L = (16, 32, 32, 48)$ ， $m = 0.05$ ， $\text{atol} = 10^{-6}$ ； $\text{mg_grid} = (2, 2, 2, 2)$ ， $\text{fine}=\text{c64}$ ；3L 另用 $8^3 \times 16$ 。

来源：实现：src/multigrid.cu:858-987、lattice_clover_multigrid.h:2908-3147。

总体流程：从规范场和 null vectors 到可验证的细层解



Multigrid production workflow

Input: $U, C, b, x^{(0)}, \{P_\ell, R_\ell, A_\ell\}, \text{dtype}_\ell, \text{MPI grid}$
Initialize: $D_0 \rightarrow S_0, \quad r_0 \leftarrow b - S_0 x^{(0)}, \quad \text{state}_\ell \leftarrow (x, r, \tilde{r}, p, v, s, t)$
for $k = 0, 1, \dots$ iterate:
 $\text{smooth}_0(r_0) \rightarrow$ if interval reached, $r_{\ell+1} \leftarrow R_\ell r_\ell$
 $x_{\ell+1} \leftarrow \text{cycle}_{\ell+1}(r_{\ell+1}), \quad x_\ell \leftarrow x_\ell + P_\ell x_{\ell+1}$
 $r_\ell \leftarrow b_\ell - A_\ell x_\ell, \quad$ coarse correction 后重置 Krylov 状态
if $\|r_0\| \leq \text{atol} \|b_0\|$ then stop; otherwise continue
Output: x_0 , full-site true residual, verification report

来源：实现：lattice_clover_multigrid.h:3236-3690, 4422-4435；图为逻辑抽象。

数据和接口：层级状态、参数协议与生命周期均显式化

对象	CUDA C++ 约定
fine vector	奇偶 Schur 向量；外部 full-site 解在桥中恢复
coarse vector	$[E_\ell, X_\ell, Y_\ell, Z_\ell, T_\ell]$ ， T 为最快布局
33 点算子	$[E, E, X, Y, Z, T]$ 、 $[2, 4, E, E, \dots]$ 、 $[2, 2, 6, E, E, \dots]$
null vector	blocked $[E, e, X, b_x, Y, b_y, Z, b_z, T, b_t]$
transition slots	每个 fine→coarse 绑定 null / nn / diag / sit 四个指针

来源：参数：pyqcu/cuda/define.py:80-98 与 cpp/cuda/qcu/include/define.h:116-130；桥：pyqcu/cuda/qcu/qcu.pyx:216-240。

扁平协议

params: int32[58]; argv: float[7];
set_ptrs: int64[100]。
_MG_LEVELn_DATA_TYPE_ 选择逐层
c64/c128；mode bits 选择平滑器、外层
和 cycle。

安全生命周期

applyInitQcu → 操作 →
params[_SET_INDEX_]+=1 →
applyEndQcu。
计数不递增会造成 scratch 槽位冲突；
verify 与 solve 共用初始化和清理边界。

Chebyshev 平滑：用一次谱尺度采样构造稳定多项式

Pseudocode —Chebyshev smoother for one MG level

Input: A_ℓ , x , r , n_{smooth}

Sample: $q \leftarrow A_\ell r$, $\hat{\rho} \leftarrow \|q\|/\|r\|$

$\lambda_{\max} \leftarrow 8\hat{\rho}$, $\lambda_{\min} \leftarrow 0.05\lambda_{\max}$, $\theta \leftarrow (\lambda_{\max} + \lambda_{\min})/2$

$p \leftarrow 0$, $\alpha \leftarrow 1/\theta$, $\beta \leftarrow 0$

for $k = 0, 1, \dots, n_{\text{smooth}} - 1$ iterate:

$p \leftarrow \alpha r + \beta p$

$x \leftarrow x + p$, $r \leftarrow r - A_\ell p$

$\beta_{k+1} \leftarrow (\delta\alpha/2)^2$, $\alpha_{k+1} \leftarrow 1/(\theta - \beta_{k+1})$

if denominator invalid, use $\beta \leftarrow 0$, $\alpha \leftarrow 0.8/\lambda_{\max}$

Output: x, r (finite update or guarded fallback)

物理/数值判断

$\hat{\rho}$ 只是局部样本；因 Schur 算子非正规，区间是保守平滑启发式，不是谱包络定理。有限性保护避免一次坏估计将 NaN 传入粗层。

实测

大格 2L: $t = 1.442\text{ s}$, full-op 真残差 6.59×10^{-7} , 与 BiCGStab 解差 8.57×10^{-6} 。

来源：实现：lattice_clover_multigrid.h:1732-1855；数据：out/qcu_mg_verify_chebyshev.json。

CA-GCR: 四阶块、双遍 MGS 与小系统回退控制通信代价

Pseudocode —communication-avoiding GCR with MG preconditioning

Input: A , $M^{-1} = \text{MG}$, $r_0 = b - Ax_0$, $s = 4$

Initialize: $q_j \leftarrow A^j r_0$ ($j = 0, \dots, s-1$)

for each block k iterate:

$\{q_j\} \leftarrow \text{two-pass modified Gram-Schmidt}(\{q_j\})$

$z_j \leftarrow M^{-1} q_j$, $w_j \leftarrow Az_j$

$G_{ij} \leftarrow \langle w_i, w_j \rangle$, $h_i \leftarrow \langle w_i, r \rangle$

$G\gamma = h$ (partial pivoting; singular block $\rightarrow$ FGMRES fallback)

$x \leftarrow x + \sum_j \gamma_j z_j$, $r \leftarrow r - \sum_j \gamma_j w_j$

if $\|r\| \leq \text{atol} \|b\|$ then stop; otherwise start next block

Output: x , block residual history

来源: 实现: `lattice_clover_multigrid.h:L5000-L5265`;

数据: `out/qcu_mg_verify_ca_gcr.json`.

为什么是 CA

多个 Krylov 方向共享一次块级 Gram/归约; 代价转移到小型 host 线性系统。双遍 MGS 控制块内失正交, 坏块显式回退而非污染解。

实测

大格 2L: $t = 24.262 \text{ s}$,

真残差 7.34×10^{-7} ;

解差 1.87×10^{-6} , 历史长度 26。

BiCGStabL: 固定 $L = 2$ 的短块递推、可靠重启和粗校正一致性

Pseudocode — BiCGStabL(2) outer solve with MG correction

Input: $S, b, x_0, L = 2, \text{ cycle interval } n_r$
 $r_0 \leftarrow b - Sx_0, \tilde{r} \leftarrow r_0, (\rho_0, \alpha, \omega) \leftarrow (1, 1, 1)$
for $k = 0, 1, \dots$ while $\|r\| > \text{atol} \|b\|$:
 for $j = 0, \dots, \min(L - 1, \text{remaining})$ do BiCG step on u_j, r_j
 $G_{ij} \leftarrow \langle r_{i+1}, r_{j+1} \rangle, g_i \leftarrow \langle r_{i+1}, r_0 \rangle$
 $G\gamma = g, x \leftarrow x + \sum_i \gamma_i u_i, r_0 \leftarrow r_0 - \sum_i \gamma_i r_{i+1}$
 if G 奇异/标量非有限: 从真残差重启; 连续 4 次后回退 BiCGStab
 if $\text{since_correction} \geq n_r$: $x \leftarrow x + PA_c^{-1} Rr$, 并重置全部 Krylov 状态
 每 8 个 block 重新计算 $r = b - Sx$ (reliable update)
Output: $x, r, \text{history}$

来源: 实现: `lattice_clover_multigrid.h:L3926-L4293`;
数据: `out/qcu_mg_verify_bicgstab_l.json`; $L = 2$ 。

关键不变量

粗校正改变 x , 旧的影子/搜索方向不再属于当前 Krylov 空间, 必须同步重置。
 $\max_{\text{iter}} \not\equiv 0 \pmod{L}$ 时只处理合法的末尾 partial block。

实测

大格 $t = 3.105 \text{ s}$;
真残差 8.02×10^{-7} ;
解差 4.85×10^{-6} 。

V/W/F/K-cycle (1/2): 同一 transfer 状态机上的四种递归策略

Pseudocode —recursive multigrid cycle dispatch

Input: (A_ℓ, r_ℓ) , P_ℓ, R_ℓ , cycle kind $\in \{V, W, F, K\}$
Pre-smooth: $x_\ell \leftarrow \text{smooth}(A_\ell, r_\ell, \nu_{\text{pre}})$
Restrict: $r_{\ell+1} \leftarrow R_\ell(r_\ell - A_\ell x_\ell)$
if ℓ is coarsest: solve $A_{\ell+1} e_{\ell+1} = r_{\ell+1}$ and return
if V: $e_{\ell+1} \leftarrow V(r_{\ell+1})$
if W: $e_{\ell+1} \leftarrow W(r_{\ell+1})$; $W(\cdot)$ second time
if F: $e_{\ell+1} \leftarrow F(r_{\ell+1})$; $V(\cdot)$ second time
if K: $e_{\ell+1} \leftarrow \text{FGMRES}(m=2, \text{right preconditioner} = K)$
Prolong/update: $x_\ell \leftarrow x_\ell + P_\ell e_{\ell+1}$, $r_\ell \leftarrow b_\ell - A_\ell x_\ell$
Post-smooth: $x_\ell \leftarrow \text{smooth}(A_\ell, r_\ell, \nu_{\text{post}})$
Output: x_ℓ and refreshed residual

来源: 递归: `lattice_clover_multigrid.h:3236-3446, 3460-3690`; 伪代码为实现逻辑抽象。

V/W/F/K-cycle (2/2): 3L 小格验证通过, 但暂不外推性能

3L 小格配置	墙钟 / s	解差	真残差
W, $E = (24, 24)$	0.229	5.91×10^{-6}	5.95×10^{-7}
F, $E = (24, 24)$	0.253	5.91×10^{-6}	5.95×10^{-7}
K, $E = (24, 24)$	0.205	5.81×10^{-6}	5.93×10^{-7}

来源: 数据: out/qcu_mg_verify_3l_*_cycle.json;
3L 为 $8^3 \times 16$ 缓存验证。

证据判断

三种递归策略均无 NaN、非法访问, 且 full-op 真残差约为 5.9×10^{-7} 。

推断 当前结果证明递归分支可运行; 样本为 $8^3 \times 16$ 缓存验证, 不能外推大格性能收益。

逐层混合精度：显式 cast kernel 保持每层存储类型和残差语义

Pseudocode —heterogeneous precision transition

Input: $\{A_\ell, P_\ell, R_\ell\}$, $\text{type}_0, \dots, \text{type}_{L-1} \in \{\text{c64}, \text{c128}\}$

Allocate: x_ℓ, r_ℓ , operators $_\ell$ in their declared level type

Restrict: $r_{\ell+1} \leftarrow R_\ell \text{cast}_{\text{type}_{\ell+1} \leftarrow \text{type}_\ell}(r_\ell)$

Coarse solve: $A_{\ell+1} e_{\ell+1} = r_{\ell+1}$ using native type $_{\ell+1}$

Prolong: $e_\ell \leftarrow P_\ell \text{cast}_{\text{type}_\ell \leftarrow \text{type}_{\ell+1}}(e_{\ell+1})$

$x_0 \leftarrow x_0 + e_0$, $r_0 \leftarrow b_0 - A_0 x_0$, reset fine Krylov state

if $\|r_0\| \leq \text{atol} \|b_0\|$ then stop; otherwise continue

Output: x_0 with full-op residual evaluated in fine precision

工程保证

不再用 `reinterpret_cast`
假设相邻层类型相同；
restrict/prolong 分别由
`multigrid_restrict_cast`
与
`multigrid_prolong_cast`
完成。

实测

fine c64/coarse c128:
 $t = 5.375 \text{ s}$, 真残差
 7.54×10^{-7} ; fine c128/coarse
c64: 真残差 6.32×10^{-7} 。

来源：实现：multigrid.cu:L199-L331、lattice_clover_multigrid.h:L4295-L4420；数据：out/*mixed*.json。

真正分布式粗格：每个 rank 只持有局部 33 点算子和局部向量

Pseudocode —distributed coarse dslash with 32-neighbour halo

Input: x_{local} , A_c^{local} , rank coordinates, periodic process grid
Map: $\{\pm e_\mu, \pm e_\mu \pm e_\nu\} \rightarrow 32$ signed neighbour directions
Pack: device kernel extracts every local boundary face into fixed $[32, E, \text{face}]$ slots
Exchange: post 32 matching receives, then 32 sends; self-neighbours use device copy
Unpack: fill one-site padded halo $[E, X+2, Y+2, Z+2, T+2]$
Apply: $y \leftarrow A_c^{\text{local}} x_{\text{local}}$ using local interior and halo boundary values
Reduce: $\langle u, v \rangle_{\text{global}} \leftarrow \text{MPI_Allreduce}(\langle u, v \rangle_{\text{local}})$
Output: y_{local} ; global scalar and relative residual are rank-consistent

分布式等价性场景	全局相对误差	结果
grid = (1, 1, 1, 2), c64	5.60×10^{-7}	PASS
grid = (2, 1, 1, 1), c128	1.03×10^{-15}	PASS
MPI MG / BiCGStabL, np=2	无死锁、无非法访问	PASS

来源: halo: cpp/cuda/qcu/include/lattice_multigrid.h:11-329 与 src/multigrid.cu:334-765; 测试: pyqcu/testing/qcu/strict/quda_comparison/coarse_mpi_smoke.py。

通信计算重叠与 transport 选择 (1/2): 正确性优先的三层回退

device-aware MPI

PYQCU_MPI_DEVICE_AWARE=auto|0|1。查询 MPI 厂商能力并检查 pointer attribute; 能力不足或显式 off 时回退 pinned-host staging, 避免把 device pointer 误交给普通 MPI。

overlap

PYQCU_MPI_OVERLAP=on|off。nonblocking halo 请求在飞行时先计算 interior, 请求完成后执行 boundary; mixed 与 homogeneous 两条路径共用事件和 stream。

NVSHMEM

CMake 以 QCU_ENABLE_NVSHMEM=ON 显式启用; 运行时 PYQCU_NVSHMEM=1, 通过 MPI communicator 对齐 PE, 使用 symmetric receive buffer 与 putmem_nbi。

选择顺序: 显式启用且 PE 对齐的 NVSHMEM → 能力与 pointer 检查通过的 CUDA-aware MPI → pinned-host staging。

来源: 实现: lattice_mpi.h:38-159、lattice_multigrid.h:227-329、CMakeLists-nv.txt:4-54; 图示为策略摘要。

通信计算重叠与 transport 选择 (2/2): 回退不改变数值 wire order

Pseudocode —safe communication policy

Input: device buffer, MPI capability, environment flags

Choose: NVSHMEM if explicitly built+requested+PE map agrees

else choose: CUDA-aware MPI if capability and pointer check pass

else choose: pinned-host pack/copy + ordinary MPI

If overlap: post transport → compute interior → wait/unpack → compute boundary

If any capability check fails: fall back without changing numerical wire order

Output: completed halo and transport name in diagnostic state

证据边界: overlap、device-aware 与 NVSHMEM 的代码路径已编译进设计; 当前 smoke 主要验证 host-staging/非重叠等价性。

NVSHMEM headers/library 与真实 CUDA-aware MPI 能力在本环境未安装或未完成实测, 故不作性能结论。

来源: 实现: lattice_mpi.h:38-159、lattice_multigrid.h:227-329、CMakeLists-nv.txt:4-54; 分区 kernel: multigrid.cu:758-765。

五项 C++ verify (1/2): 从 transfer 到 normal operator 的统一契约

Pseudocode —complete C++ multigrid verify() contract

Input: initialized hierarchy, $\{P_\ell, R_\ell, A_\ell\}$, fine Clover data

1. Projection: $\eta_c \rightarrow P\eta_c \rightarrow RP\eta_c$, $\|RP\eta_c - \eta_c\|/\|\eta_c\|$
 2. Transfer: $\langle Pc, v \rangle$ versus $\langle c, Rv \rangle$
 3. Galerkin: $\eta_c \rightarrow P\eta_c \rightarrow S(P\eta_c) \rightarrow RSP\eta_c$ versus $A_c\eta_c$
 4. Preconditioned operator: $D_{ee}x_e - \kappa H_{eo}x_o$ and $D_{oo}x_o - \kappa H_{oe}x_e - Sx_o$
 5. Normal operator: $\langle x, S^\dagger Sx \rangle$ versus $\langle Sx, Sx \rangle$, positivity
- Aggregate: overall $\leftarrow (1) \wedge (2) \wedge (3) \wedge (4) \wedge (5)$
- Output: 0 = all pass, 1 = diagnostic fail, 2 = bridge/runtime error

来源: 接口: `cpp/cuda/qcu/src/apply_clover_multigrid_verify.cu:14-125`、
`lattice_clover_multigrid.h:2908-3147`; 伪代码概括五项诊断与返回语义。

五项 C++ verify (2/2): 五项误差均通过, 接口返回 status=0

诊断	实测相对误差	状态	status=0
projection	2.32×10^{-7}	PASS	
restriction / prolongation	4.70×10^{-7}	PASS	
Galerkin	6.14×10^{-7}	PASS	
preconditioned operator	3.71×10^{-8}	PASS	
normal operator	0	PASS	

验收结论

五项接口在同一初始化/清理边界内聚合执行; 最大实测相对误差为 6.14×10^{-7} , 所有诊断返回 status=0。

来源: 接口: apply_clover_multigrid_verify.cu:14-125、lattice_clover_multigrid.h:2908-3147; Cython: qcu.pyx:216-240; 数值记录: 本会话日志。

统一大格结果：所有新求解路径达到 full-op 真残差验收线

路径	墙钟 / s	历史长度	参考解差异	full-op 真残差
Chebyshev smoother	1.442	17	8.57×10^{-6}	6.59×10^{-7}
CA-GCR ($s = 4$)	24.262	26	1.87×10^{-6}	7.34×10^{-7}
BiCGStabL ($L = 2$)	3.105	58	4.85×10^{-6}	8.02×10^{-7}
mixed c64→c128	5.375	86	5.14×10^{-6}	7.54×10^{-7}

量纲与口径

墙钟是 CUDA 调用同步后的秒；真残差重新作用 full Wilson+Clover 算子，分母为同一 RHS 范数。解差只作为同一 PyQCU BiCGStab 参考的数值一致性指标。

来源：数据：out/qcu_mg_verify_chebyshev.json、qcu_mg_verify_ca_gcr.json、qcu_mg_verify_bicgstab_l.json、qcu_mg_verify_mixed_c64_c128.json；统一 $16 \times 32 \times 32 \times 48$ 、 $m = 0.05$ 、 $\text{atol} = 10^{-6}$ 。

判断

确证 代码分支和数值闭环成立；**推断** CA-GCR 当前代价明显更高，不能因为通信规避的理论目标而宣称实测加速。

回归闸门与边界测试 (1/2): 构建、桥接与求解路径均通过

闸门	证据	结果
CUDA C++ build	bash./build.sh 成功链接 libqcu.so, float/double 模板均完成	PASS
Cython bridge	bash./install.sh 成功; verifyCloverMultigridQcu 导出	PASS
default regression	run_all.py: 真残差、MG/reference、component quality	GREEN (3/3)
breakdown guard	test_mg_breakdown.py: 零粗算子不污染解	1 passed

来源: 回归: pyqcu/testing/qcu/strict/quda_comparison/out/regression.json; 构建命令来自仓库 AGENTS.md。

回归闸门与边界测试 (2/2): MPI、递归 cycle 与资源边界均有记录

闸门	证据	结果
coarse MPI equivalence	np=2, 多种 process grid、精度、overlap 请求	PASS
3L cycle matrix	W/F/K 无 NaN、无非法访问, full-op 残差约 5.9×10^{-7}	PASS

异常资源与失败边界

P100×16 的一次运行受显存资源不足影响, 未计为算法失败; NVSHMEM 专用编译未在本机完成。

CUDA 编译仍有既有的 dynamic initialization / unused variable 警告, 但不影响构建退出码和本轮测试。

来源: MPI driver: run_qcu_mg_mpi.py、coarse_mpi_smoke.py; 3L 数据: out/qcu_mg_verify_3l_{w,f,k}_cycle.json。

当前风险：正确性已闭环，但性能声明仍需拆分证据等级

项目	已知事实	不应外推的结论
3L W/F/K CA-GCR	$8^3 \times 16$ 缓存真实运行，数值通过 大格真残差 7.34×10^{-7}	不能外推大格 3L 性能收益 当前 24.3 s 不能代表健康 平台通信收益
混合精度	2L/3L 单 rank，2-rank smoke 通过	尚未覆盖任意层数/动态误差 预算
device-aware MPI	capability query 与安全回退代码已 实现	当前环境未证明 MPI 真正 接收 device pointer
overlap	interior/boundary kernel 顺序和事 件已实现	尚无独立 kernel overlap speedup 曲线
NVSHMEM	opt-in CMake、PE 对齐、 put/quiet/barrier 路径存在	未在 NVSHMEM 环境编 译/运行
BiCGStabL	固定 $L = 2$ ，有重启/回退	不是任意可配置 L 的通用 实现

来源：状态依据：out/qcu_mg_verify_*.json、lattice_mpi.h、lattice_multigrid.h；“未验证”遵循证据分级，不以静态代码替代实测。

交付结论：功能闭环已完成，性能进入平台验收阶段

本阶段可交付

产物	状态
CUDA C++ MG 算法族	PASS
per-level mixed precision	PASS
rank-local coarse + 32 halo	PASS
overlap/device-aware/NVSHMEM API	PASS / 未实测
C++/Cython 五项 verify	PASS
回归与本报告	PASS

当前边界

确证 算法、混合精度、分布式粗格和五项 verify 已形成数值闭环。

未验证 device-aware MPI、NVSHMEM 真实通信能力与大格 3L 性能仍待专用平台补测。

下一步验收对象

固定 RHS 与 wire order，对比三种 transport 的通信/计算时间；覆盖大格、多 rank、逐层混合精度及非整除 L 。

结论：实现工作完成；
性能与部署条件转入健康通信栈验收；
当前数值结论不变。

来源：交付：本报告 PDF 与 TeX；状态：差异检查、工作树与标签。

代码审计与算法验证

PyQCU CUDA MG

下一步验收：在健康通信栈上量化性能并扩展边界

Pseudocode —next reproducible acceptance loop

Select: healthy CUDA-aware MPI node + NVSHMEM toolchain

Build: compile off/on matrix for device-aware and NVSHMEM

Measure: t_{pack} , t_{MPI} , t_{interior} , t_{boundary} and bandwidth

Compare: host-staging vs device-aware vs NVSHMEM, same wire order and RHS

Stress: large 3L, multiple ranks, mixed precision, non-multiple L

Accept: all five verify + full-op residual + no deadlock or illegal access

Output: platform-qualified performance report

性能结论必须在对应通信栈上实测；当前报告只对数值正确性和安全回退作确证。

来源：验收计划：由本轮通信接口、MPI smoke 与 full-op residual 闭环推导；性能数据尚未验证。

附录：关键 mode bits 与可复现命令

语义	bit	接口选择
FGMRES/GCR	1	--gcr
MR smoother	2	--smoothermr
Chebyshev	4	--smootherchebyshev
CA-GCR	8	--smootherca-gcr
W/F/K cycle	16 / 32 / 64	--cyclew f k
BiCGStabL	128	--bicgstab-l

Pseudocode —reproducible invocation contract

```
Prepare: source ./env.sh
Build: bash ./build.sh; bash ./install.sh
Run solver: python pyqcu/testing/qcu/strict/quda_comparison/run_qcu_mg.py --levels 2 --E 12
Run 3L: --levels 3 --E 24 --coarse-E 24 --cycle w|f|k
Run mixed: --fine-dtype c64 --coarse-dtypes c128
Run MPI: mpirun -np 2 python pyqcu/testing/qcu/strict/quda_comparison/coarse_mpi_smoke.py
Check: python pyqcu/testing/qcu/strict/quda_comparison/run_all.py; git diff --check
```

来源：CLI: pyqcu/testing/qcu/strict/quda_comparison/run_qcu_mg.py、run_qcu_mg_mpi.py; mode 定义: pyqcu/cuda/define.py:88-98。

附录：证据索引与最终状态

证据域	文件/结果
求解器族与 cycle	out/qcu_mg_verify_{chebyshev,ca_gcr,bicgstab_l}.json;
混合精度	out/qcu_mg_verify_3l_{w,f,k}_cycle.json out/qcu_mg_verify_mixed_c64_c128.json; out/qcu_mg_mixed_c128_c64.json
分布式粗格	pyqcu/testing/qcu/strict/quda_comparison/coarse_mpi_smoke.py; MPI 运行输出
C++ verify	apply_clover_multigrid_verify.cu; lattice_clover_multigrid.h:2908-3147
回归闸门 构建/桥接	out/regression.json, GREEN (3/3) build.sh、install.sh; 导出符号 verifyCloverMultigridQcu

结果可复查，未验证项已显式标注；
代码未提交、未推送，最终仅创建本地注释标签。